

Laborator 2 - Heartbeat

Inițializare

Inițializez sistemul distribuit prin pornirea fiecărui proces pe mașini diferite, conectându-mă la un grup multicast (ex. 230.0.0.0 pe portul 4446). Fiecare proces trimite periodic heartbeat-uri pentru a anunța că este activ.

Verificare

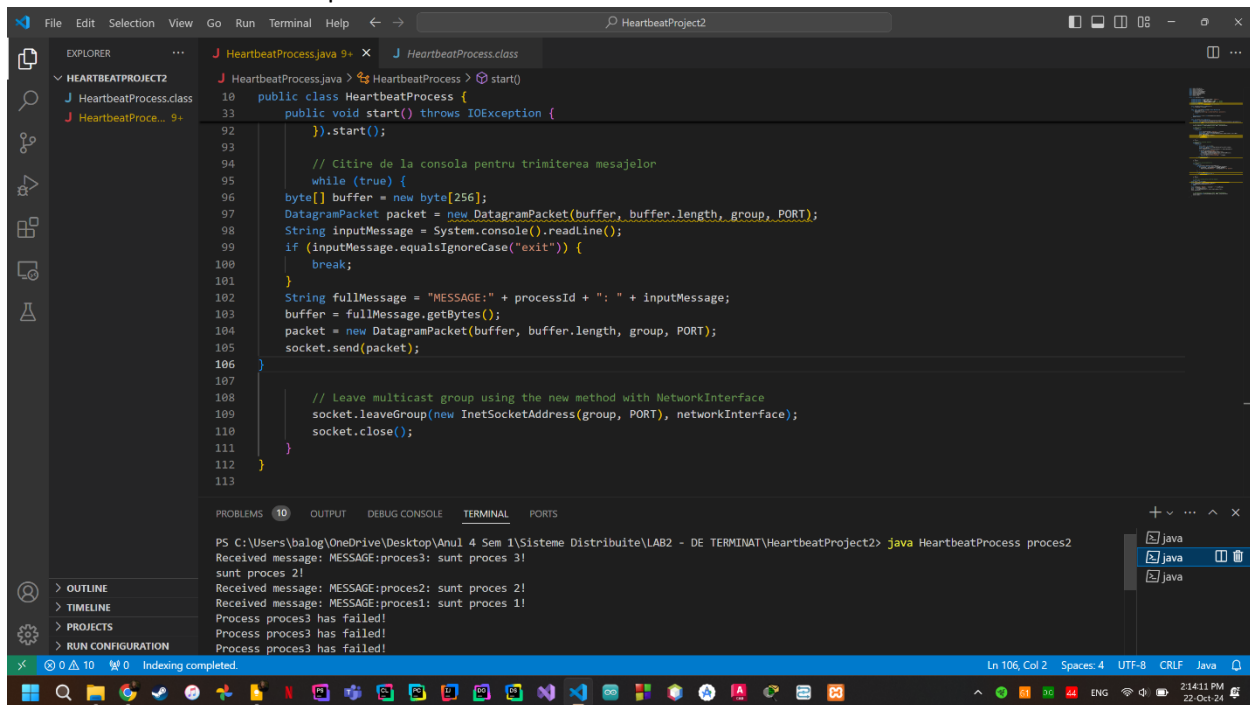
Monitorizez celelalte procese prin primirea heartbeat-urilor. Dacă nu primesc un heartbeat de la un proces timp de 5 secunde, consider că acel proces a eșuat și afișez un mesaj corespunzător.

Simularea defectării

Simulez defectarea unui proces prin apăsarea Ctrl + C sau prin introducerea comenzii exit. Celelalte procese vor detecta lipsa heartbeat-urilor și vor anunța că procesul respectiv a căzut.

Închiderea proceselor

Procesele pot fi închise manual cu Ctrl + C sau programat prin comanda exit, permițându-mi să curăț resursele înainte de a mă opri.



The screenshot shows an IDE window titled "HeartbeatProject2". The Explorer pane on the left shows the project structure with "HEARTBEATPROJECT2" containing "HeartbeatProcess.class" and "HeartbeatProcess.java". The main editor displays the "HeartbeatProcess.java" file with the following code:

```
10 public class HeartbeatProcess {
33     public void start() throws IOException {
92     }
93
94     // Citire de la consola pentru trimiterea mesajelor
95     while (true) {
96         byte[] buffer = new byte[256];
97         DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, PORT);
98         String inputMessage = System.console().readLine();
99         if (inputMessage.equalsIgnoreCase("exit")) {
100             break;
101         }
102         String fullMessage = "MESSAGE:" + processId + ": " + inputMessage;
103         buffer = fullMessage.getBytes();
104         packet = new DatagramPacket(buffer, buffer.length, group, PORT);
105         socket.send(packet);
106     }
107
108     // Leave multicast group using the new method with NetworkInterface
109     socket.leaveGroup(new InetSocketAddress(group, PORT), networkInterface);
110     socket.close();
111 }
112
113 }
```

The TERMINAL pane at the bottom shows the command prompt output:

```
PS C:\Users\balog\OneDrive\Desktop\Anul 4 Sem 1\Sisteme Distribuite\LAB2 - DE TERMINAT\HeartbeatProject2> java HeartbeatProcess proces2
Received message: MESSAGE:proces3: sunt proces 3!
sunt proces 2!
Received message: MESSAGE:proces2: sunt proces 2!
Received message: MESSAGE:proces1: sunt proces 1!
Process proces3 has failed!
Process proces3 has failed!
Process proces3 has failed!
```

The status bar at the bottom indicates "Ln 106, Col 2, Spaces: 4, UTF-8, CRLF, Java".

```
File Edit Selection View Go Run Terminal Help < -> HeartbeatProject2

EXPLORER
HEARTBEATPROJECT2
HeartbeatProcess.java
HeartbeatProcess.class
HeartbeatProcess... 9+

HeartbeatProcess.java
HeartbeatProcess > start()
10 public class HeartbeatProcess {
33 public void start() throws IOException {
92 }
93
94 // Citire de la consola pentru trimiterea mesajelor
95 while (true) {
96 byte[] buffer = new byte[256];
97 DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, PORT);
98 String inputMessage = System.console().readLine();
99 if (inputMessage.equalsIgnoreCase("exit")) {
100 break;
101 }
102 String fullMessage = "MESSAGE:" + processId + ": " + inputMessage;
103 buffer = fullMessage.getBytes();
104 packet = new DatagramPacket(buffer, buffer.length, group, PORT);
105 socket.send(packet);
106 }
107
108 // Leave multicast group using the new method with NetworkInterface
109 socket.leaveGroup(new InetSocketAddress(group, PORT), networkInterface);
110 socket.close();
111 }
112 }
113 }

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS
rive\Desktop\Anul 4 Sem 1\Sisteme Distribuite\LAB2 - DE TERMINAT\HeartbeatProject2> javac HeartbeatProcess.java
PS C:\Users\ba\log\OneDrive\Desktop\Anul 4 Sem 1\Sisteme Distribuite\LAB2 - DE TERMINAT\HeartbeatProject2> java HeartbeatProcess proces1
Received message: MESSAGE:proces3: sunt proces 3!
Received message: MESSAGE:proces2: sunt proces 2!
sunt proces 1!
Received message: MESSAGE:proces1: sunt proces 1!
Process proces3 has failed!
Process proces3 has failed!

Ln 106, Col 2 Spaces: 4 UTF-8 CRLF Java
```

```
File Edit Selection View Go Run Terminal Help < -> HeartbeatProject2

EXPLORER
HEARTBEATPROJECT2
HeartbeatProcess.java
HeartbeatProcess.class
HeartbeatProcess... 9+

HeartbeatProcess.java
HeartbeatProcess > start()
10 public class HeartbeatProcess {
33 public void start() throws IOException {
92 }
93
94 // Citire de la consola pentru trimiterea mesajelor
95 while (true) {
96 byte[] buffer = new byte[256];
97 DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, PORT);
98 String inputMessage = System.console().readLine();
99 if (inputMessage.equalsIgnoreCase("exit")) {
100 break;
101 }
102 String fullMessage = "MESSAGE:" + processId + ": " + inputMessage;
103 buffer = fullMessage.getBytes();
104 packet = new DatagramPacket(buffer, buffer.length, group, PORT);
105 socket.send(packet);
106 }
107
108 // Leave multicast group using the new method with NetworkInterface
109 socket.leaveGroup(new InetSocketAddress(group, PORT), networkInterface);
110 socket.close();
111 }
112 }
113 }

PROBLEMS 10 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Process proces3 has failed!
Process proces3 has failed!
Process proces3 has failed!
Process proces3 has failed!
Process proces3 has failed!
Process proces3 has failed!
Received message: MESSAGE:proces3: am revenit, sunt proces 3!

Ln 106, Col 2 Spaces: 4 UTF-8 CRLF Java
```

Codul complet

```
import java.io.IOException;

import java.net.DatagramPacket;

import java.net.InetAddress;

import java.net.InetSocketAddress;

import java.net.MulticastSocket;

import java.net.NetworkInterface;

import java.util.HashMap;

import java.util.Map;


public class HeartbeatProcess {


    private static final String MULTICAST_GROUP = "230.0.0.0";

    private static final int PORT = 4446;

    private static final int HEARTBEAT_INTERVAL = 2000; // 2 secunde

    private static final int TIMEOUT_INTERVAL = 5000; // 5 secunde

    private String processId;

    private Map<String, Long> lastHeartbeat = new HashMap<>();


    public HeartbeatProcess(String processId) {

        this.processId = processId;

    }


    public static void main(String[] args) throws IOException {

        if (args.length != 1) {

            System.out.println("Usage: java HeartbeatProcess <processId>");

            return;

        }


        HeartbeatProcess process = new HeartbeatProcess(args[0]);

        process.start();

    }

}
```

```

public void start() throws IOException {

    InetAddress group = InetAddress.getByName(MULTICAST_GROUP);

    NetworkInterface networkInterface = NetworkInterface.getByInetAddress(InetAddress.getLocalHost());

    MulticastSocket socket = new MulticastSocket(PORT);

    // Join multicast group using the new method with NetworkInterface
    socket.joinGroup(new InetSocketAddress(group, PORT), networkInterface);

    // Thread pentru trimiterea heartbeat-urilor
    new Thread(() -> {

        while (true) {

            try {

                String heartbeatMessage = "HEARTBEAT:" + processId;

                byte[] buffer = heartbeatMessage.getBytes();

                DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, PORT);

                socket.send(packet);

                Thread.sleep(HEARTBEAT_INTERVAL);

            } catch (Exception e) {

                e.printStackTrace();

            }

        }

    }).start();

    // Thread pentru ascultarea mesajelor
    new Thread(() -> {

        while (true) {

            try {

                byte[] buffer = new byte[256];

                DatagramPacket packet = new DatagramPacket(buffer, buffer.length);

                socket.receive(packet);

                String message = new String(packet.getData(), 0, packet.getLength());

                if (message.startsWith("HEARTBEAT:")) {

                    String otherProcessId = message.split(":")[1];

                    lastHeartbeat.put(otherProcessId, System.currentTimeMillis());

                }

            } catch (Exception e) {

                e.printStackTrace();

            }

        }

    }).start();
}

```

```

    } else if (message.startsWith("MESSAGE:")) {

        System.out.println("Received message: " + message);

    }

} catch (IOException e) {

    e.printStackTrace();

}

}

}).start();

```

// Thread pentru verificarea defectării

```

new Thread() -> {

    while (true) {

        long currentTime = System.currentTimeMillis();

        for (Map.Entry<String, Long> entry : lastHeartbeat.entrySet()) {

            if (currentTime - entry.getValue() > TIMEOUT_INTERVAL) {

                System.out.println("Process " + entry.getKey() + " has failed!");

            }

        }

        try {

            Thread.sleep(1000);

        } catch (InterruptedException e) {

            e.printStackTrace();

        }

    }

}).start();

```

// Citire de la consola pentru trimiterea mesajelor

```

while (true) {

    byte[] buffer = new byte[256];

    DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, PORT);

    String inputMessage = System.console().readLine();

    if (inputMessage.equalsIgnoreCase("exit")) {

        break;

    }

    String fullMessage = "MESSAGE:" + processId + ": " + inputMessage;

```

```
buffer = fullMessage.getBytes();

packet = new DatagramPacket(buffer, buffer.length, group, PORT);

socket.send(packet);
}

// Leave multicast group using the new method with NetworkInterface
socket.leaveGroup(new InetSocketAddress(group, PORT), networkInterface);

socket.close();
}
}
```